

Today 2:39 PM

 ai(1).py Code	 board(1).py Code	 game(1).py Code
 main(1).py Code	 README(1).md Document	 requirements(1).txt Document

help me with the tasks, run me through the reasons for the changes made too.

Worked for 1m 9s

Analyzed

Analysis errored

I went through the supplied project against the README tasks and built a corrected copy. I kept the existing module structure (ai.py , board.py , game.py , main.py) rather than replacing it with a different implementation, as the assignment asks.  README

Corrected project:  [Download the completed Battleship project](#)

What was wrong originally

Task 1 — Coordinate handling

The project is supposed to use one coordinate representation internally. The original AI did not do that:

 Python

```
pos = random.choice(options)
...
return f"{pos[0] + 1},{pos[1] + 1}"
```

The AI selected a zero-based coordinate such as (0, 0) , converted it to "1,1" , and then game.py converted "1,1" directly back to (1, 1) instead of (0, 0) .  ai

So the coordinate effectively moved by one.

Change I made: coordinates are now:

- **0-based tuples internally:** (0, 0) through (5, 5)
- **1-based row,col only at the user interface**

The AI now returns the tuple directly rather than converting it to a string.

Why this is better:

 Plain text

```
User enters 3,3
  ↓
game converts it to (2,2)
  ↓
Board checks (2,2)
```

And for the AI:

 Plain text

```
AI chooses (2,2)
  ↓
Board receives (2,2)
  ↓
Only when printing → "3,3"
```

That gives us one unambiguous internal representation.

The README specifically requires this consistency across input, ships, player shots and AI shots.

 README

Task 2 — Multiple ships and proper sinking

Originally `Board` only had:

 Python

```
self.ships = set()
self.shots = set()
```

and:

 Python

```
return pos in self.ships
```

That tells us whether a cell contains *some* ship, but it doesn't tell us **which ship was hit** or whether that individual ship has been sunk.  `board`

Change

I added:

`</>` Python

```
self.ship_cells = []  
self.hits = set()
```

When a ship is placed, its cells are stored separately:

`</>` Python

```
self.ship_cells.append(cells)
```

Now `fire()` can determine:

1. Was the shot a hit?
2. Which ship was hit?
3. Has that ship now been completely hit?

For example, suppose we have:

`</>` Plain text

```
Ship A = {(2,2), (2,3), (2,4)}  
Ship B = {(4,4), (5,4)}
```

If the player fires:

`</>` Plain text

```
3,3 → HIT  
3,4 → HIT  
3,5 → HIT
```

the third shot produces:

`</>` Plain text

```
HIT!  
You sank a ship.
```

But the game continues because Ship B is still alive.

Then:

 Plain text

```
5,5 → HIT  
6,5 → HIT  
You sank a ship.  
You sank the fleet.
```

That satisfies the requirement to distinguish **sinking one ship** from **sinking the entire fleet**.

 README

I also changed the setup to contain two ships on each board rather than only one.

An important additional bug I fixed

There was a state-tracking mix-up in `game.py`.

Originally, the player's input was checked against:

 Python

```
self.player.shots
```

But `self.player.shots` represents shots **made against the player's board** — in other words, AI shots.

The player's shots are actually stored on:

 Python

```
self.enemy.shots
```

So I changed the repeated-shot check to use the enemy board.

The same applies to:

 Python

```
Ship cells remaining
```

It now calculates remaining enemy ship cells using the enemy board's shots.

This is important because otherwise the game can incorrectly think that the player has already fired at a location when it was actually the AI that fired there.

Task 3 — Better AI

The original AI simply did:

`</>` Python

```
options = all untried cells
random.choice(options)
```

So it had no concept of a successful hit. `</>` ai

The assignment asks for the AI to preferentially look around a cell after it hits something.  [README](#)

Change

The new AI accepts:

`</>` Python

```
choose(last_hit=None)
```

If it has just hit at:

`</>` Plain text

```
(2, 2)
```

it considers:

`</>` Plain text

```
(1,2)
(3,2)
(2,1)
(2,3)
```

first.

It filters out:

- coordinates outside the board

- coordinates it already tried

Only if there are no suitable adjacent cells does it go back to random untried cells.

Why?

Imagine a ship occupies:

</> Plain text

```
. . . . .  
. . X X X  
. . . . .
```

Once the AI hits the first `X`, shooting randomly across the entire board wastes information.

Checking adjacent cells gives it a reasonable chance of finding the rest of the ship.

It isn't a sophisticated Battleship solver, but it implements exactly the requested behaviour without making the AI unnecessarily complicated.

AI repeated shots

The AI keeps:

</> Python

```
self.tried = set()
```

and every selected coordinate is added to it.

So:

</> Python

```
if pos not in self.tried
```

is always enforced.

I also added the exhausted-board case:

</> Python

```
if len(self.tried) == self.size * self.size:
```

```
return None
```

That prevents `random.choice([])` from crashing when there are no legal moves left.

The README explicitly requires both behaviours.  [README](#)

Task 4 — Shot-level feedback

The original game did the AI conversion itself:

 Python

```
ai_pos = self.ai.choose()

try:
    ar, ac = map(int, ai_pos.split(","))
    player_pos = (ar, ac)
```

and then performed hit detection separately. 

That makes it very easy for AI candidate selection and actual shot resolution to become mixed together.

Change

Now the flow is:

 Plain text

```
AI chooses a coordinate
    ↓
game prints the actual shot
    ↓
Board.fire() resolves it
    ↓
game prints HIT or MISS
    ↓
game prints SANK only if appropriate
```

The AI itself doesn't print hit/miss messages.

This means merely **considering a candidate coordinate** cannot accidentally generate gameplay feedback.

That's what the README means by requiring feedback exactly once for an actual shot.  [README](#)

Testing I performed

I checked the important cases from the assignment:

Test	Result
Ship coordinate is recognised as a hit	Pass
Empty coordinate is recognised as a miss	Pass
Multiple ships	Pass
Individual ship damage	Pass
One ship sinks before the other	Pass
Entire fleet sinking	Pass
Repeated player shot prevented	Pass
AI never repeats a coordinate	Pass
AI handles no remaining choices	Pass
AI prefers adjacent cells after hit	Pass
AI falls back when adjacent cells unavailable	Pass
Invalid text input	Pass
Out-of-range coordinates	Pass
Python compilation	Pass

The final sinking test produced the expected sequence:

</> Plain text

```
HIT!  
HIT!  
HIT!  
You sank a ship.  
HIT!  
HIT!
```

You sank a ship.
You sank the fleet.

I also verified that an empty coordinate produces a miss and a known ship coordinate produces a hit.

How to explain the changes if you're asked

A good way to explain your work is:

Task 1: I standardised the internal coordinate representation to zero-based (row, column) tuples. User input remains one-based because that is more natural for the player. The original AI converted its coordinates to one-based strings and then the game treated them as zero-based again, causing an offset.

Task 2: I separated the fleet into individual ships and added hit tracking. This allows the program to know when one particular ship is sunk while still keeping the game running until every ship is sunk.

Task 3: I changed the AI so that after a hit it considers the four neighbouring cells first. It still tracks every coordinate it has tried, so it cannot shoot the same cell twice, and it safely handles the situation where no cells remain.

Task 4: I separated AI selection from shot resolution. The AI only chooses a coordinate; the game/board resolves the shot and produces the feedback. This prevents internal candidate selection from generating false hit or miss messages.

Those explanations map directly to the four tasks in your README.  [README](#)

One thing to remember for the assignment: the README specifically asks for a **before-change gameplay video**, an **after-change gameplay video**, and the **complete LLM chat-history link**.

 [README](#)

For the before video, the most useful defect to demonstrate is the original AI coordinate mismatch: the AI chooses a zero-based cell, turns it into a one-based string, and `game.py` then interprets that string as zero-based. That gives you a concrete bug to point at rather than just showing that the old game is incomplete. `</>`

the ai in `_ai_turn` seems to forget a hit after a miss? also please verify that off-board/overlap cells and out of bound positions aren't accepted.

Worked for 30s

Yes. Your suspicion was correct.

1. The AI was forgetting a hit after a miss

In `_ai_turn()` , the previous version did this:

`</>` Python

```
else:
    print("AI missed.")
    self.ai_last_hit = None
```

So the sequence was:

`</>` Plain text

```
AI hits (2,2)
↓
ai_last_hit = (2,2)
↓
AI tries a neighbour
↓
MISS
↓
ai_last_hit = None ← problem
↓
next choice becomes random
```

I changed it so a miss **does not clear the last successful hit**.

Now:

`</>` Plain text

```
AI hits (2,2)
↓
AI tries neighbour
↓
MISS
↓
still remembers (2,2)
↓
tries another untried neighbour
```

The hit is only cleared when the ship is actually sunk. That is the sensible interpretation of the requirement to preferentially consider nearby untried cells after a hit. The README specifically calls for adjacent targeting after a hit. [🔗 README](#)

I tested this directly: after forcing an AI miss, `ai_last_hit` remained `(1, 1)` .

2. Off-board ship cells are now rejected

Previously, `place_ship()` simply did:

`</>` Python

```
self.ships.update(cells)
```

so something like:

`</>` Python

```
board.place_ship({(-1, 0), (0, 0)})
```

could enter an invalid board state.

It now validates every ship cell before adding it:

`</>` Python

```
if not (0 <= pos[0] < cls.SIZE and 0 <= pos[1] < cls.SIZE):  
    raise ValueError("Position is outside the board.")
```

So all of these are rejected:

`</>` Plain text

```
(-1, 0)  
(6, 0)  
(0, -1)  
(0, 6)
```

For a 6×6 board, the only valid internal coordinates are:

`</>` Plain text

```
0 ≤ row ≤ 5  
0 ≤ col ≤ 5
```

3. Overlapping ships are now rejected

I also added:

`</>` Python

```
if self.ships.intersection(cells):  
    raise ValueError("Ship cells overlap an existing ship.")
```

So this is invalid:

`</>` Python

```
board.place_ship({(1, 1), (1, 2)})  
board.place_ship({(1, 2), (2, 2)})
```

because (1, 2) belongs to both ships.

This is important because otherwise `ship_cells` could contain overlapping ships and the individual sinking logic would become ambiguous.

4. `Board.fire()` now rejects out-of-bounds positions too

This is an additional protection.

Previously, `_player_turn()` checked bounds, but someone calling:

`</>` Python

```
board.fire((99, 99))
```

directly could bypass that validation.

Now `fire()` calls the same position validator first.

So invalid positions cannot enter through either:

- the user interface
- the AI
- direct `Board` calls

The AI already limits its generated positions to the board bounds, but this gives the board itself a defensive boundary.

5. The AI still can't repeat shots

I re-tested the AI exhaustion case:

Plain text

```
2x2 board  
→ 4 unique shots  
→ fifth choose() returns None
```

So it doesn't generate duplicates and doesn't crash when there are no choices left.

Current validation

I ran tests covering:

- AI remembers a hit after a miss
- AI chooses another neighbour rather than immediately going random
- AI doesn't repeat shots
- AI doesn't generate off-board coordinates
- `Board.place_ship()` rejects negative coordinates
- `Board.place_ship()` rejects coordinates equal to `SIZE`
- `Board.place_ship()` rejects overlapping ships
- `Board.fire()` rejects out-of-bounds coordinates
- Python compilation of all modules

All targeted tests passed.

The updated `Board` validation is now concentrated in `_validate_pos()`, so there isn't a separate, potentially inconsistent bounds check in every operation. The relevant validation and overlap checks are in the corrected `board.py`.

[!\[\]\(51c8b64a0f70f0b96d4cbd0a65299579_img.jpg\) Download the updated Battleship project](#)

One subtle point: **a miss should not erase the hit target, but sinking should.** If the AI hits a ship at (2,2), misses (2,3), then hits (2,4), it should continue using the hit information. Once (2,2), (2,4), and the rest of that ship's cells have sunk the ship, clearing the target is appropriate so the AI can return to general searching.

In `game.py`, `_player_turn` returns `False` for both rejected input and a normal shot, so after an invalid entry or a repeated shot, the AI still fires. Also, my README requires quitting to be tested too, but I don't see that handled?

Worked for 46s

Correct. There were two separate issues.

`_player_turn()` was conflating two outcomes

Previously:

`</>` Python

```
return False
```

was used for both:

1. Rejected input

- invalid format
- out-of-range coordinate
- repeated shot

2. Valid shot, game continues

Then `run()` couldn't distinguish them:

`</>` Python

```
if self._player_turn(row):  
    return  
  
self._ai_turn()
```

So even this:

`</>` Plain text

```
> abc  
Use row,col.
```

would proceed to the AI.

I changed the return values to explicit states:

`</>` Plain text

```
"rejected" → don't give the AI a turn  
"continue" → valid shot, AI gets a turn  
"won"      → player won, end game
```

The important part of `run()` is now effectively:

`</>` Python

```
player_result = self._player_turn(raw)

if player_result == "rejected":
    continue

if player_result == "won":
    return

if self._ai_turn():
    ...
```

So the turn sequence is now:

`</>` Plain text

```
Invalid input
  ↓
reject
  ↓
same player's turn again

Valid shot
  ↓
resolve shot
  ↓
AI gets a turn

Player sinks fleet
  ↓
game ends
```

I tested invalid input, out-of-range input, a normal miss, and a repeated shot. All behaved correctly.

Quitting

Your README explicitly lists **quitting** among the required tests.  [README](#)

The quit handling was actually already present in the original `run()` :

`</>` Python

```
if raw == "q":
    return
```

So you were right to look for it, but it wasn't something that needed to be added from scratch.

I've kept that behaviour and verified it:

 Plain text

```
Battleship
```

```
Your shots are coordinates like 2,3.
```

```
Ship cells remaining: 5
```

```
> q
```

The game exits immediately, and importantly **the AI does not get a turn after quitting.**

Updated project

 [Download the updated Battleship project](#)

The final turn-handling design now satisfies the README's requirements around invalid input, repeated shots, normal shots, winning, and quitting.  [README](#)

could you create tests for all the testing requirements from the
readme please?

Worked for 54s

Yes. I created a complete `unittest` test suite based directly on the README's required testing list. The README says to test hits, misses, repeated shots, sinking one ship, sinking all ships, AI repeated-shot prevention, adjacent targeting after a hit, invalid coordinates, and quitting.  [README](#)

I also included the additional validation cases we've been discussing:

- off-board ship placement
- overlapping ships
- out-of-bounds `Board.fire()`
- AI not generating off-board positions
- AI retaining a hit after a miss
- rejected player input not giving the AI a turn
- repeated player shots not giving the AI a turn
- exactly one hit/miss feedback message

Test file

The new file is:

 Plain text

```
test_battleship.py
```

It contains **21 tests** across board behaviour, AI behaviour, game turn handling, and feedback.

Run them with:

 Bash

```
python -m unittest -v test_battleship.py
```

I ran the entire suite myself:

 Plain text

```
Ran 21 tests in 0.005s
```

```
OK
```

Download

 [Download the Battleship project with the complete test suite](#)

I also added the test-running instructions to `README.md`.

One small cleanup point: the ZIP currently contains Python's generated `__pycache__` files because the tests were executed before packaging. They aren't needed for submission. If you want, I can produce a **clean submission ZIP with only the source files, README, requirements, and tests**.